

The Critical Question

Which tasks can't slip?

Pulse Works conference room, late afternoon. A whiteboard is covered in sticky notes representing project activities. Saki has drawn lines between them with a red marker. Tetsu leans against the far wall, arms crossed.

Ren (蓮) "Okay, we've got the **schedule** drafted. Every **activity** has an owner and a duration. So we're good, right?"

Saki (咲希) "We have a list. We don't have a plan. Have you identified which **dependencies** create the longest chain? That chain is your **critical path** (the longest sequence of dependent activities — it determines the shortest possible project duration)."

→ The *critical path* reveals which activities directly control the project's end date. Any delay on a critical-path activity delays the entire project.

Ren (蓮) "Wait — so not every task matters equally for the timeline?"

Saki (咲希) "Exactly. Some activities have **float** (the amount of time they can slip without delaying the project). Critical-path activities have zero float. Let me show you with a **network diagram** (a graphical map of activity dependencies)."

→ A *network diagram* makes dependency chains visible. It is the foundation for calculating the critical path and float values.

Tetsu (哲) "The rendering engine, the API integration, and the pilot deployment — those are sequential. None of them can start until the prior one finishes."

Saki (咲希) "Correct. And if we run a **forward pass** (calculating the earliest possible start and finish for each activity), we'll see exactly how tight the timeline is."

→ A *forward pass* works left-to-right through the network diagram, adding durations to find the earliest each activity can begin and end.

Ren (蓮) "So the rendering engine is on the critical path. And Tetsu is the only one who can build it. That's... not great."

Saki (咲希) "No. It isn't. But recognizing it is the first step."

She said it without a trace of condescension. Ren looked at her — really looked — and for a moment forgot what he was going to say next.

Critical Path

DEFINITION The longest sequence of dependent activities determining the shortest project duration.

COMPONENTS Sequential activities with zero float, earliest/latest start-finish dates, total project duration.

RELATED TERMS **dependency**, float, network diagram

EXAMPLE In the XR project, rendering engine → API integration → pilot deployment forms the critical path. Slipping any one of these pushes the launch date.

Float

DEFINITION The amount of time an activity can be delayed without delaying the project.

COMPONENTS Total float (delay without affecting project end), free float (delay without affecting successor start).

RELATED TERMS critical path, **schedule**, forward pass

EXAMPLE The analytics dashboard has 8 days of float — it can start a week late without affecting the launch, because it runs in parallel to the critical path.

Network Diagram

DEFINITION A graphical representation of activity dependencies.

COMPONENTS Nodes (activities), arrows (dependencies), path sequences, start/end points.

RELATED TERMS **dependency**, **WBS**, critical path

EXAMPLE Saki's whiteboard drawing — sticky notes as activities, red lines as dependencies — is a hand-drawn network diagram.

Forward Pass

DEFINITION A technique to calculate early start and early finish dates.

COMPONENTS Early start (ES), early finish (EF), activity duration, predecessor relationships.

RELATED TERMS network diagram, critical path, **schedule**

EXAMPLE Running a forward pass on the XR network diagram reveals the rendering engine must start by Week 2 or the entire 9-month timeline slips.

Review Corner

1 The Missing Pieces

Ren (蓮) "Tetsu, the scope statement says the client supplies 3D assets. But we haven't confirmed formats or file sizes — that's a gap in our **requirements**."

Tetsu (哲) "Define 'compatible format.' There are at least four we'd need to support."

→ Requirements gathering (Unit 7): Vague requirements create hidden work. Specific technical questions surface real constraints early.

2 Breaking It Down

Saki (咲希) "Your **WBS** lists 'Build Mobile Apps' as a single **work package**. That's two platforms, three feature sets, and QA — it needs decomposition."

Ren (蓮) "Fair point. If we can't estimate it, it's not broken down enough."

→ Scope definition & WBS (Unit 6): A work package should be small enough to estimate and assign. If you can't, decompose further.

3 Sequencing Check

Tetsu (哲) "The **schedule** shows QA starting in parallel with development. Is that a real **dependency**, or are we hoping?"

→ Schedule management (Unit 8): Every dependency in the schedule should be validated. Assumed parallelism hides risk.